



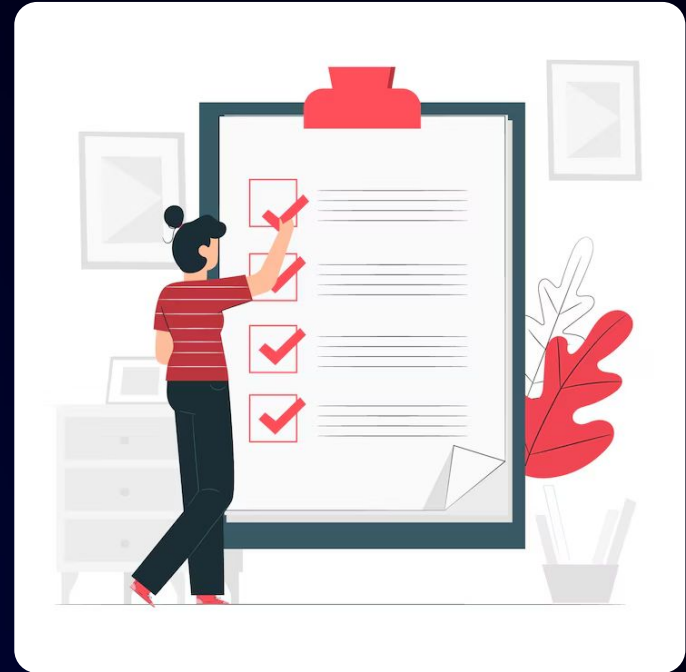
Responsive Design, CSS Grid & Basic Animations

Learning Objectives

- Make web pages responsive across different devices
- Understand the basics of responsive design
- Learn and apply media queries (core skill)
- Understand breakpoints and the mobile-first approach
- Get introduced to CSS Grid for layout design
- Learn basic CSS transitions for smoother UI interactions
- Understand the introduction to keyframe animations
- Build a responsive mini layout through a guided project

Prerequisites

- Basic understanding of programming
- Understanding HTML
- Understanding CSS Fundamentals, Structure and behavior and layout and Flexbox

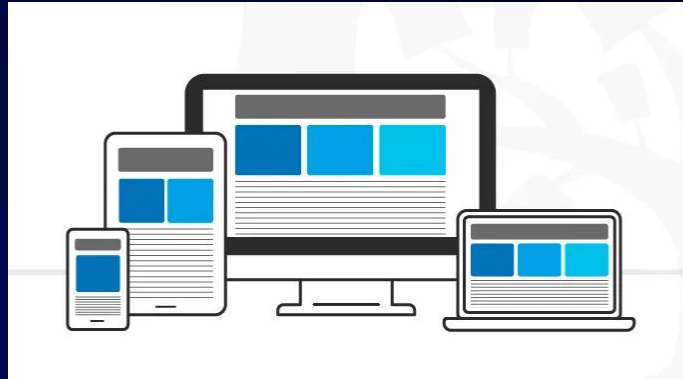




Responsive Design Basics

What is Responsive Web Design?

- Responsive Web Design (RWD) is an approach to Web Design and Development that aims to create websites that provide an optimal viewing and user experience across a wide range of devices and screen sizes.
- For example Smartphones, Tablets, and various other Devices,
- it is important for websites to be adaptable and accessible on different platforms



What happens when your website is not

Poor User Experience

Users may find it difficult to navigate your site on mobile devices, leading to frustration and higher bounce rate

Lost Traffic

Visitors using mobile devices might leave your site if it doesn't display properly, reducing your overall traffic.



Lower Search Rankings

Search engines like Google may rank your site lower in search results if it's not mobile-friendly.

Reduced Conversions

A non-responsive site can hinder user actions like signing up, purchasing, or contacting you, leading to fewer conversions

Core Principles of Responsive Web Design



01

Fluid Grids

Use proportional sizing for page elements.

02

Flexible Media

Resize images and media proportionally.

03

Media Queries

Apply styles based on device characteristics.

04

Mobile First

Design for smaller screens first, then enhance for larger ones

05

Responsive Navigation

Adapt navigation for better usability on all devices.

06

Adjustable Typography

Ensure text remains readable on any device.

Advantages of using Responsive Web design



Improves User Experience

Consistent and optimized across all devices.



Increases Mobile Traffic

Accommodates growing smartphone and tablet use.



Boosts SEO

Google favors mobile-friendly sites.



Cost-Efficient

No need for separate mobile and desktop sites.



Faster Load Times

Optimized media improves speed.



Future-Proofs

Adapts to new devices and screen sizes.



Enhances Conversion Rates

Better user satisfaction leads to higher sales.

Disadvantages of using Responsive web design



Complex Design Process

Requires careful planning and design.



Slower Performance on Older Devices

High resource usage can affect performance.



Limited Control

Some elements might not look perfect on all devices.



Longer Development Time

More complex to build and test.



Potential Compatibility Issues

Might not work perfectly on all browsers and devices.

Hands-On | Responsive Design Basics

Pop Quiz Question

Q. Which of the following is one of the core principles of Responsive Web Design?

A

Static image

B

Fluid Grids

Pop Quiz Solution

Q. Which of the following is one of the core principles of Responsive Web Design?

A

Static image

B

Fluid Grids



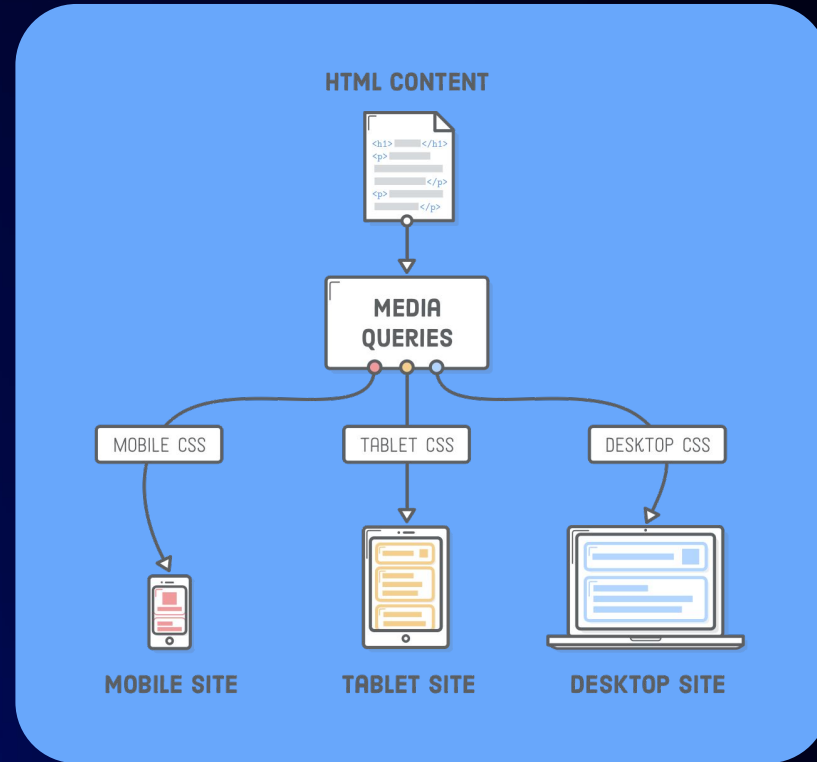
Media Queries (Core Skill)

What is Media Queries in CSS

- Media queries in CSS allow you to define different styles for different device types, screen sizes, and orientations.
- This is particularly useful for creating responsive web designs that adapt to different devices and screen sizes.
- Media queries are written using the **@media** rule, followed by one or more conditions in parentheses.
- The conditions can be based on various criteria, including the device width, device height, orientation, and pixel density.

Syntax -

```
@media [media-type] ([media-feature]) {  
  /* Styles! */  
}
```



Anatomy of a Media Query

@media screen (min-width: 320px) and (max-width: 768px)

AT-RULE

MEDIA TYPE

MEDIA FEATURE

OPERATOR

MEDIA FEATURE

► @media

► Media types

► Media features

► Operators

- The **@media** rule is the starting point for defining media queries in CSS.
- In CSS, **@media** is an at-rule that allows you to apply styles based on specific conditions, typically related to the characteristics of the user's device or browser.
- Media queries using **@media** are used to create responsive designs that adapt to different screen sizes, resolutions, and device capabilities.
- It is used to define a block of styles that should only be applied under certain conditions
- (i.e. based on media query conditions).

Syntax -

```
@media [media-type] ([media-feature]) {  
  /* Styles! */  
}
```

- The media type is a keyword that specifies the type of device or media being targeted.
- There are several media types that can be used in a media query, including
 - **all**: The media type targets all devices and media types.
 - **screen**: This media type targets devices with a screen, such as desktops, laptops, tablets, and smartphones.
 - **print**: This media type targets device that are used for printing, such as printers and PDF generators.
 - **speech**: This media type targets speech-based devices, such as screen readers.

Media Type

Syntax -

```
@media [media-type] ([media-feature]) {  
  /* Styles! */  
}
```

```
@media screen and ([media-feature]) {  
  /* Styles...  
}
```

*Here is an example of a media query that targets screen

Media Feature

A media feature is used to test specific characteristics of the device or viewport, such as width, height, orientation, and resolution.

Here are some common media features that can be used in a media query:

- **width:** specifies the width of the viewport.
- **height:** specifies the height of the viewport.
- **Orientation:** Specifies the orientation of the device.
- **Aspect-ratio:** specifies the aspect ratio of the viewport.
- **Resolution:** specifies the resolution of the device.
- **Color:** Specifies the number of bits per color channel.
- **hover:** specifies whether the device has a pointing device or not.
- **pointer:** specifies the type of pointing device available.

Media Feature

```
@media screen and ([media-feature]) {  
  /* Styles ...  
}
```

```
@media screen and (max-width: 600px) {  
  /* Styles for screens with a maximum width of  
  600px */  
}
```

*Here is an example of a media query that targets screens with a maximum width of 600 pixels

Operators

- Logical operators such as **and**, **or**, and **not** can be used to combine multiple media features or media types to create more complex conditions for when styles should be applied.

- **Here are some of the operators**

- **and**: Combines conditions; all must be true.
- **or**: Combines conditions; at least one must be true.
- **not**: Negates a condition; the condition must be false.

```
@media screen (min-width: 320px) and  
(max-width: 768px) {  
  .element {  
    /* Styles! */  
  }  
}
```

```
@media print and ( not(color) ) {  
  body {  
    background-color: none;  
  }  
}
```

```
@media screen (prefers-color-scheme: dark),  
(min-width 1200px) {  
  .element {  
    /* Styles! */  
  }  
}
```

Media queries for different screens

Device type	Example	Screen Size	Media Query	
Extra Small Devices	iPhone SE, Samsung Galaxy S5 Mini, older smartphones	< 576px	@media (max-width: 575px) { ... }	
Small Devices	iPhone 8, Samsung Galaxy S8, Google Pixel 2	576px - 767px	@media (min-width: 576px) and (max-width: 767px) { ... }	
Medium Devices	iPad, Samsung Galaxy Tab, iPad Mini	768px - 991px	@media (min-width: 768px) and (max-width: 991px) { ... }	

Media queries for different screens

Device type	Example	Screen Size	Media Query	
Large Devices	Small laptops, large tablets, iPad Pro	992px - 1199px	@media (max-width: 575.98px) { ... }	
Extra Large Devices	Desktops, larger laptops, MacBook Pro	1200px - 1599px	@media (min-width: 992px) and (max-width: 1199.98px) { ... }	

Hands-On | Media Queries (Core Skill)

Pop Quiz Question



Q. Which of the following media features can be used to apply styles only when the viewport width is 600px or less?

A

max-width

B

min-width

Pop Quiz Solution

Q. Which of the following media features can be used to apply styles only when the viewport width is 600px or less?

A

max-width

B

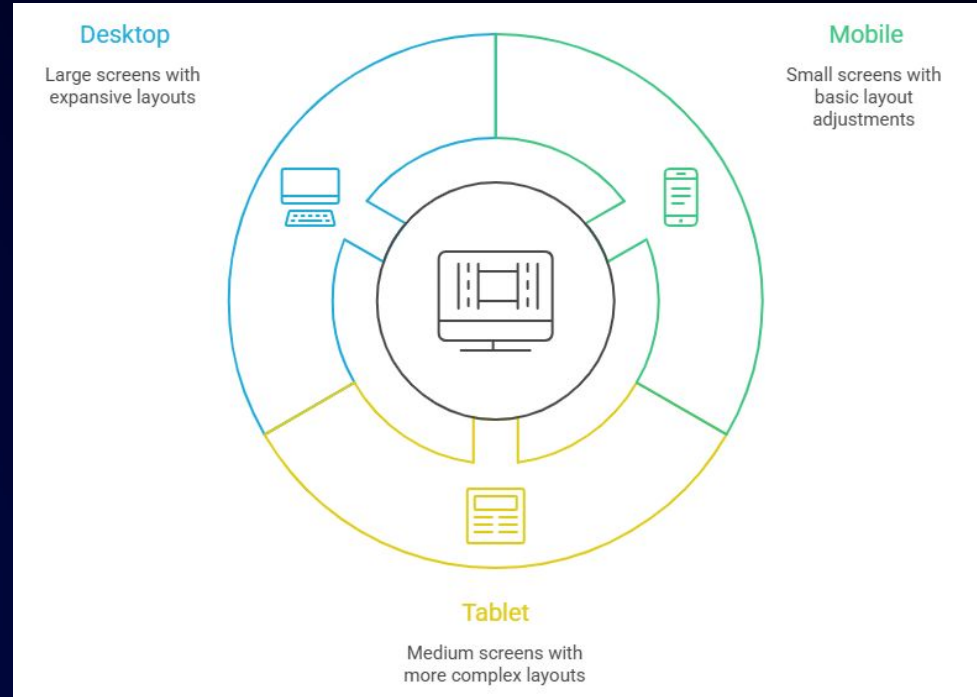
min-width



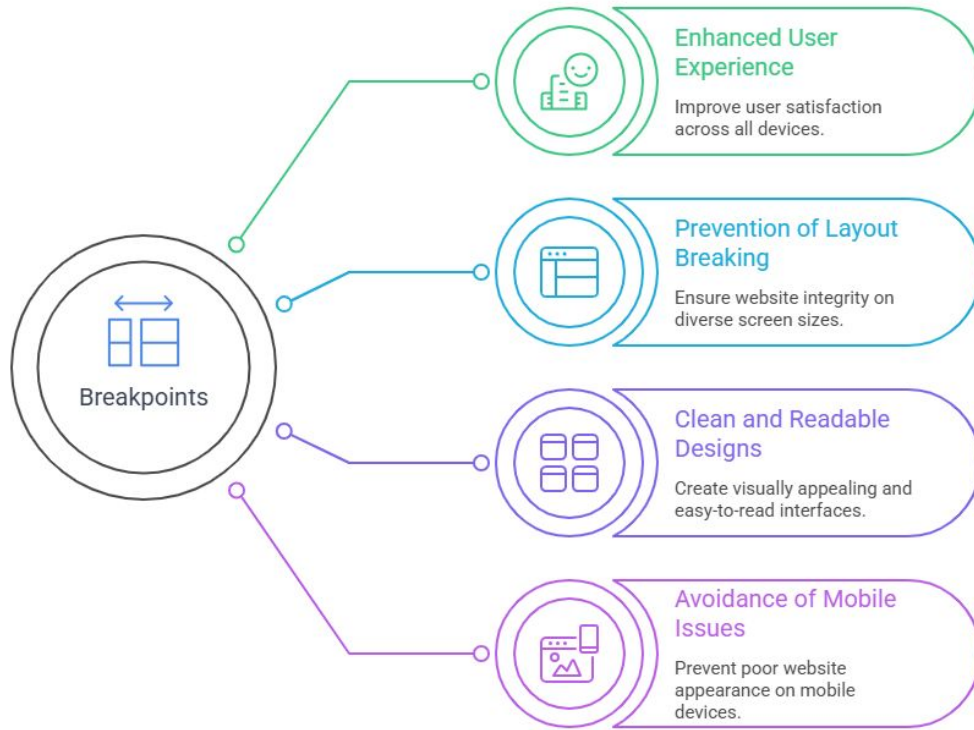
Breakpoints & Mobile-First Approach

What are Breakpoints?

- Breakpoints are screen size checkpoints
- They help adjust layout for different devices
- Used with media queries



Why Do We Need Breakpoints?



What is Mobile-First Approach?



How Mobile-First Works

How to implement a mobile-first approach?



Best Practices



Hands-On | Breakpoints & Mobile-First Approach

Pop Quiz SOLUTION

Q. What is the main idea of mobile-first design?

A

**Design for small screens first,
then scale up**

B

Design for desktop first

Pop Quiz SOLUTION

Q. What is the main idea of mobile-first design?

A

**Design for small screens first,
then scale up**

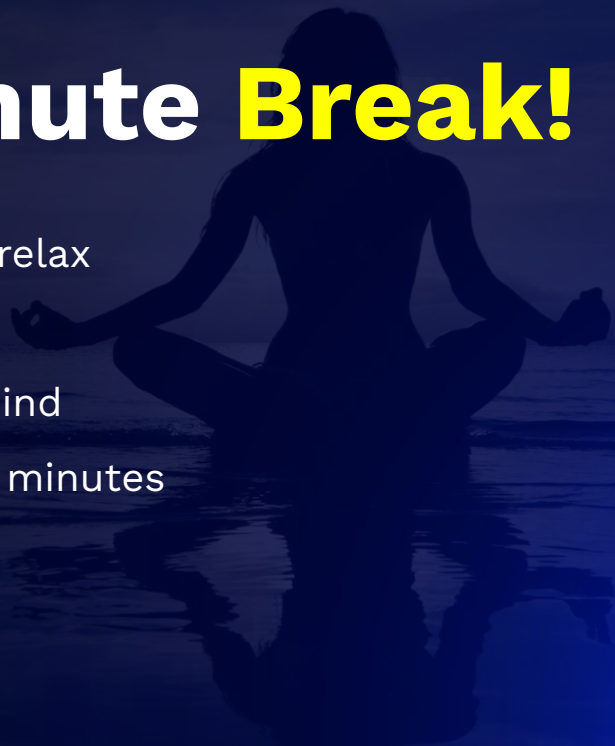
B

Design for desktop first



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

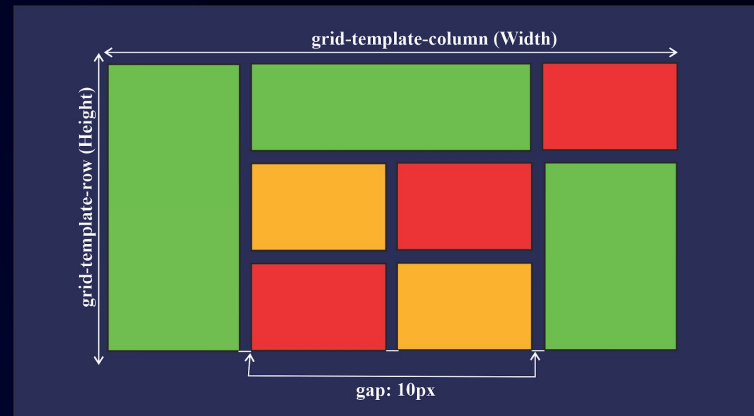




Introduction to CSS Grid

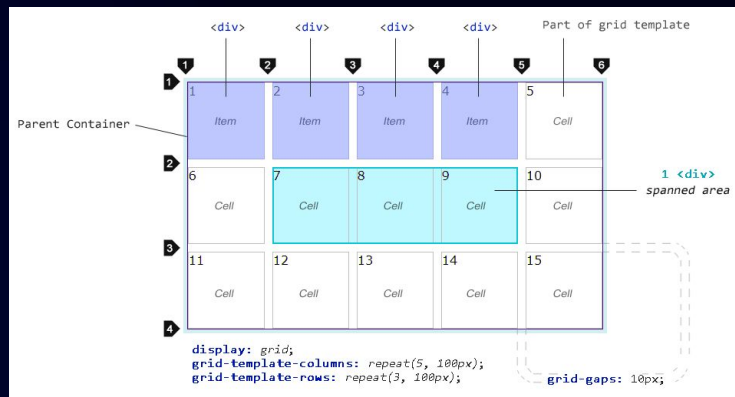
Introduction to CSS Grid Layout

- CSS Grid is a powerful layout system in CSS.
- Allows for complex, two-dimensional layouts.
- It simplifies the creation of responsive designs.
- Works well for both large and small-scale layouts.
- Provides a robust way to structure web pages.
- CSS Grid enhances design flexibility and control.



Understanding Grid Layout System

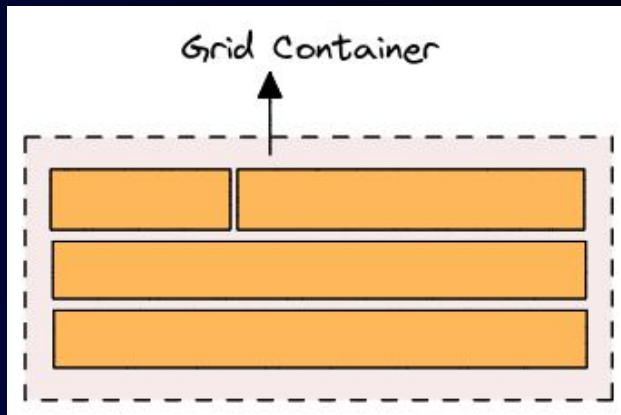
- The grid layout consists of rows and columns.
- It helps create complex designs easily.
- Provides more control over alignment and spacing.
- Ideal for creating dynamic and responsive layouts.
- Supports both horizontal and vertical alignment.
- Simplifies the handling of overlapping elements.
- Makes it easier to create consistent designs.
- Useful for page layouts, dashboards, and more.



Grid Container

The element on which `display: grid` is applied. It's the direct parent of all the grid items. In this example container is the grid container.

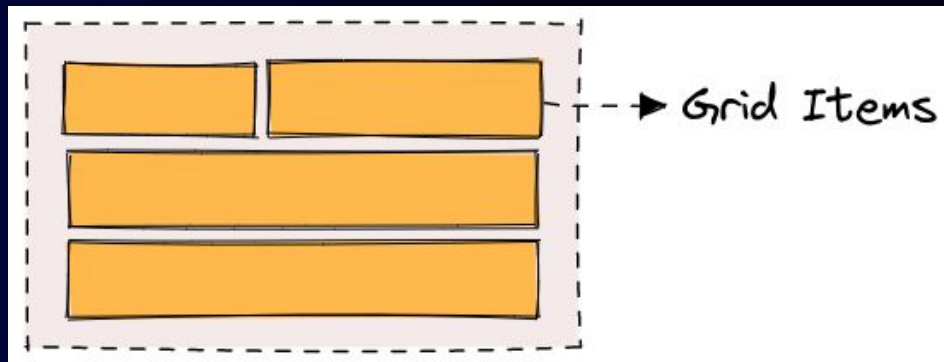
```
<div class="container">
  <div class="item item-1"> </div>
  <div class="item item-2"> </div>
  <div class="item item-3"> </div>
</div>
```



Grid Item

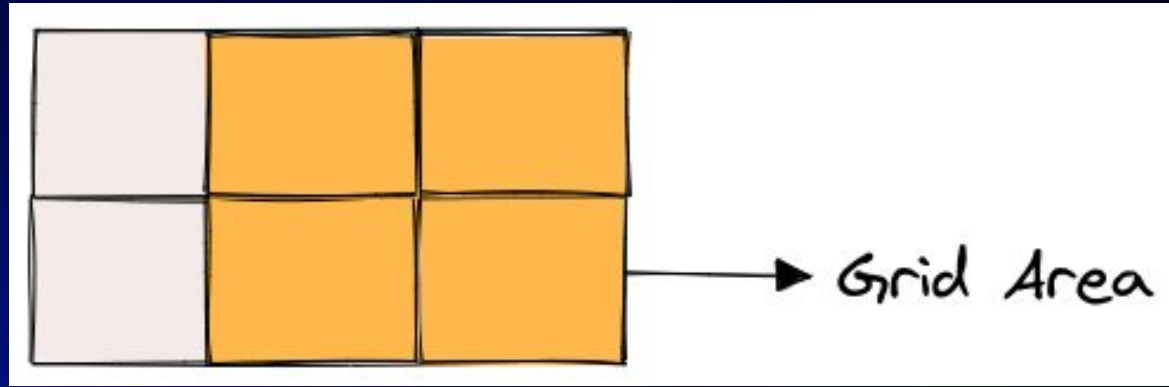
The children (i.e. direct descendants) of the grid container. Here the item elements are grid items, but sub-item isn't.

```
<div class="container">
  <div class="item"> </div>
  <div class="item">
    <p class="sub-item"> </p>
  </div>
  <div class="item"> </div>
</div>
```



Grid Area

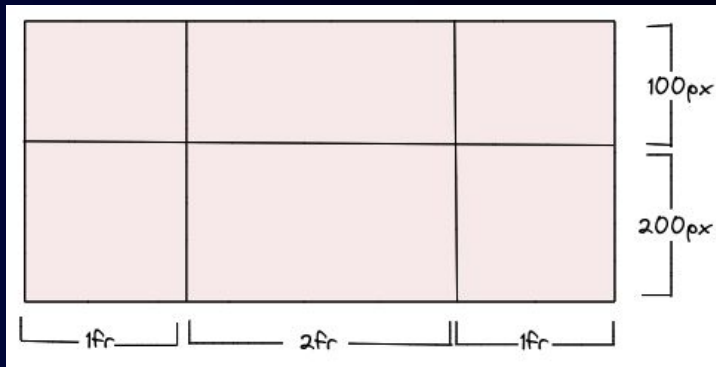
The total space surrounded by four grid lines. A grid area may be composed of any number of grid cells. Here's the grid area between row grid lines 1 and 3, and column grid lines 1 and 3.



grid-template-columns & grid-template-rows

`grid-template-columns` and `grid-template-rows` are used to define the size and position of columns and rows in a CSS grid. These properties allow you to create a flexible and responsive grid layout that can adjust to different screen sizes and content.

```
.container {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr;
  grid-template-rows: 100px 200px;
}
```



Hands-On | Introduction to CSS Grid

Pop Quiz

Q. What property is used to define the columns in a CSS Grid layout?

A

`grid-template-columns`

B

`grid-auto-columns`

Pop Quiz

Q. What property is used to define the columns in a CSS Grid layout?

A

`grid-template-columns`

B

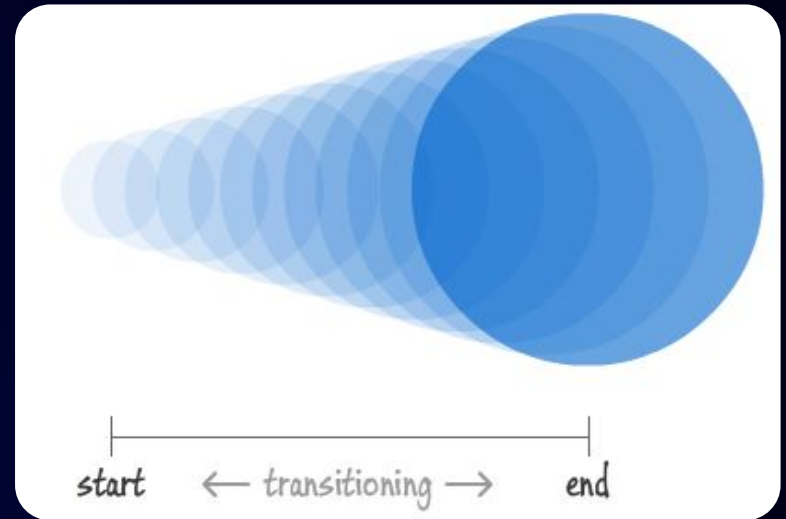
`grid-auto-columns`



Transitions (Basic)

Introduction to CSS Transitions

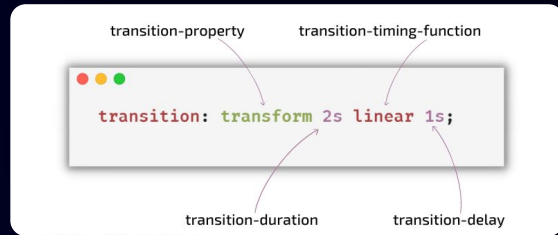
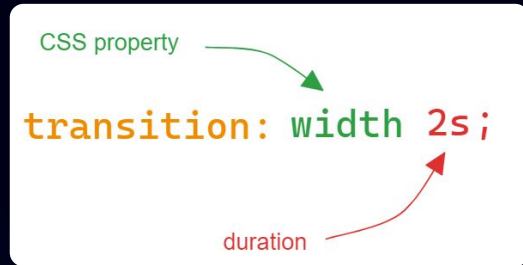
- CSS transitions enable you to control the speed of animation as you modify CSS attributes.
- CSS Transition consists of -
 - When animation will trigger
 - When animation will start (delay)
 - Duration of transition
 - How will transitions run?



How to use CSS Transitions

- To create a transition effect, you must specify two things:
 - the **CSS property (transition-property)** you want to add an effect to
 - the **duration (transition-duration)** of the effect
 - the **transition-timing function** (optional)
 - the **transition delay** (optional)

***Note:** If the duration part is not specified, the transition will have no effect, because the default value is 0.



CSS Transition Syntax

The css property you want to animate (eg. width, height, background, transform)

The time the transition takes to complete (e.g., 2s, 500ms)

```
selector {  
  transition: property duration timing-function delay  
}
```

(Optional) The speed curve of the transition (e.g., ease, linear, ease-in, ease-out, ease-in-out)

(Optional) The delay before the transition starts (e.g., 1s, 200ms).

Importance of CSS Transitions



Enhanced User Experience

Provides smooth, natural changes, reducing jarring effects



Improved Visual Appeal

Adds modern, polished aesthetics to websites



Engagement and Interactivity

Keeps users engaged and encourages interaction



Usability and Accessibility

Clarifies state changes and provides intuitive visual cues



Performance and Efficiency

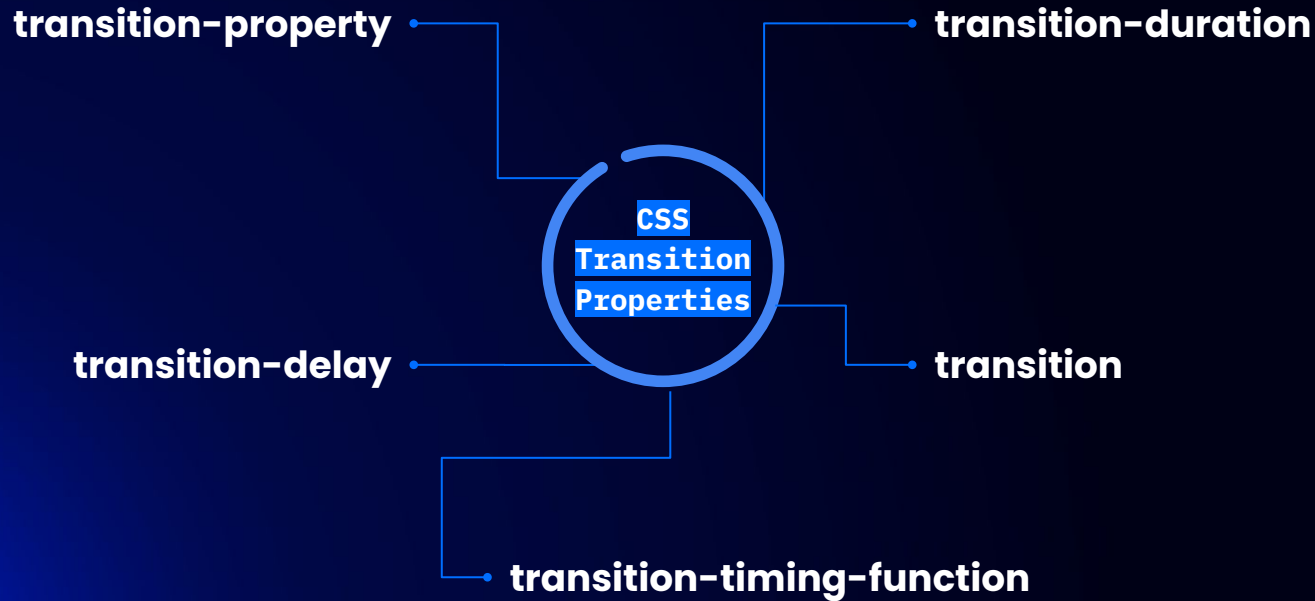
Optimized by browsers for better performance



Simpler Code and Maintenance

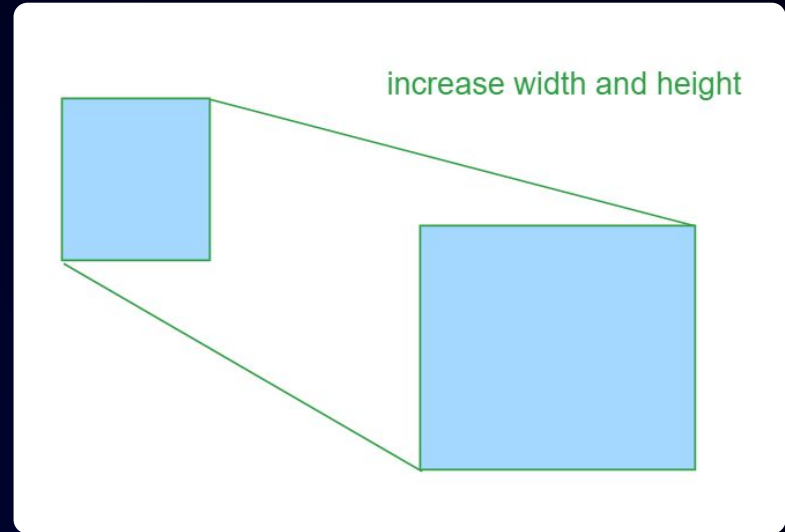
Easier to implement and maintain than JavaScript animations

Transition Properties



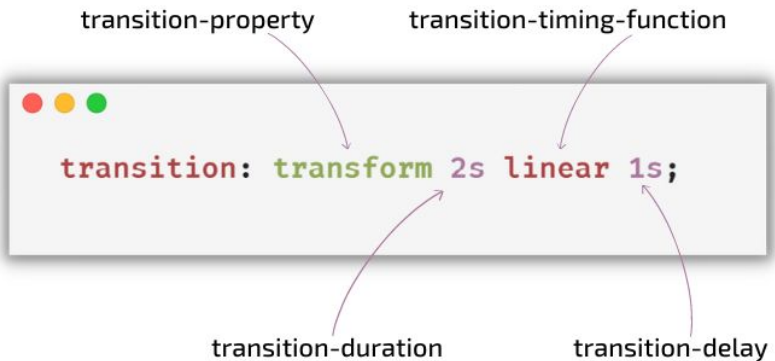
Transition Properties

- Specifies which properties will be transitioned.
- You can specify a comma-separated list of properties, or use the value all to transition all properties.
- The transition property can have a value of -
 - **all:** Default value. All properties will get a transition effect
 - **property:** No property will get a transition effect
 - **initial:** Sets this property to its default value
 - **inherit:** Inherits this property from its parent element



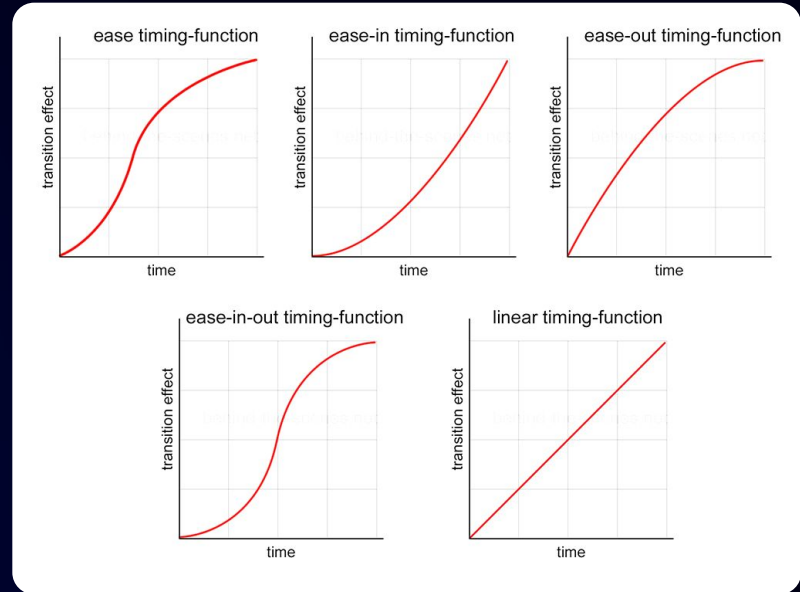
Transition-shorthand

- A shorthand property for setting the four transition properties into a single property
- The transition property is a shorthand property for:
 - transition-property
 - transition-duration
 - transition-timing-function
 - transition-delay



Transition-Timing-Function

- Specifies the timing function used for the transition.
- There are several predefined timing functions, such as
 - **ease:** effect with a slow start, then fast, then end slowly(default)
 - **ease-in:** effect with a slow start
 - **ease-out:** effect with a slow end
 - **ease-in-out:** effect with a slow start and end
 - **linear:** effect with the same speed from start to end



Hands-On | Transitions (Basic)

Pop Quiz

- Q. In CSS Transition property, what is the default value of the 'transition-timing-function' property

A

ease-in

B

ease

Pop Quiz

- Q. In CSS Transition property, what is the default value of the 'transition-timing-function' property

A

ease-in

B

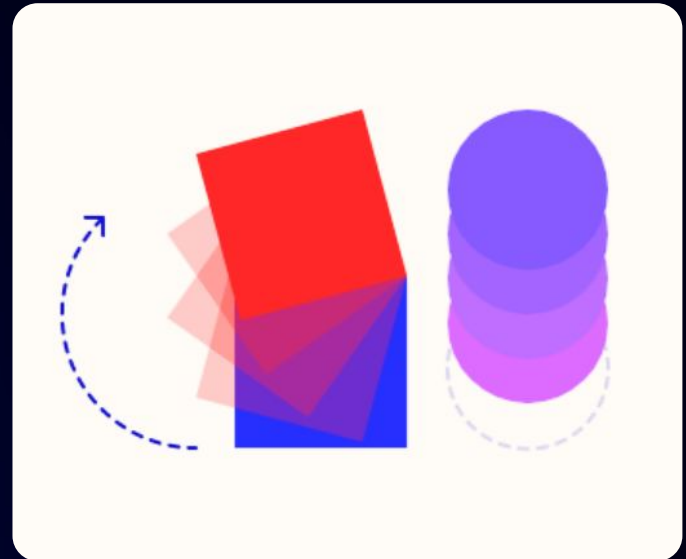
ease



Keyframe Animations (Intro)

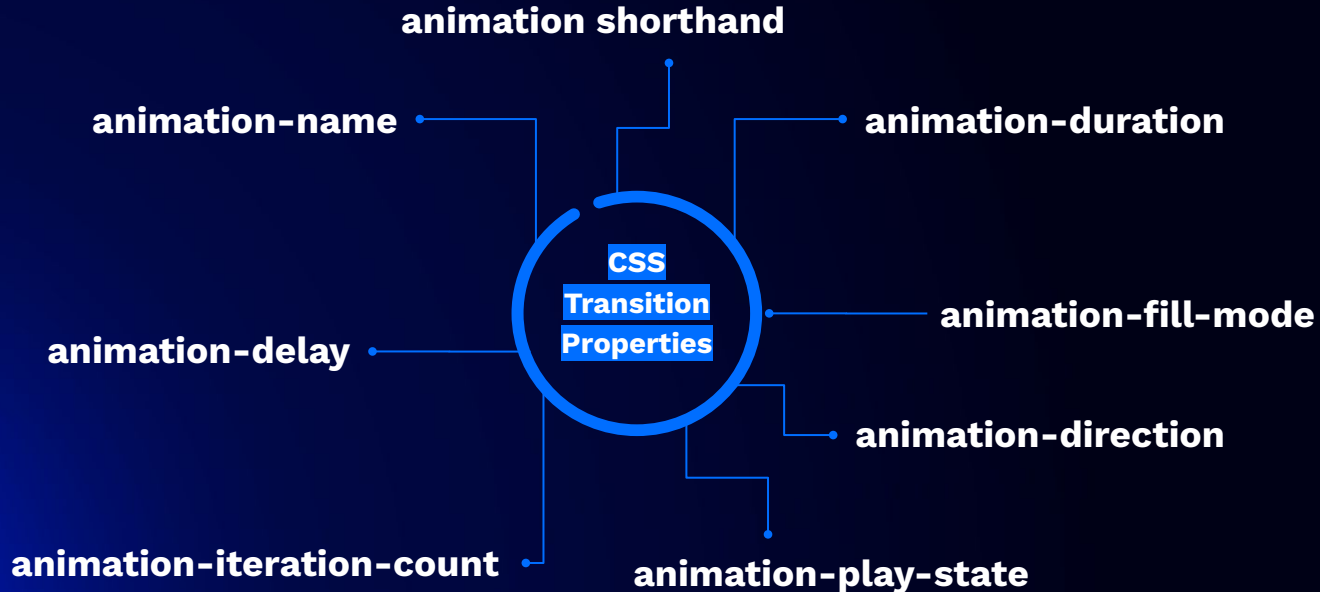
Introduction to CSS Animations

- CSS allows animation of HTML elements without using JavaScript!
- With CSS animations, it becomes feasible to animate transitions between different CSS style configurations.
- These animations typically comprise two parts:
 - a **collection style declaration** that outlines the CSS animation,
 - of **keyframes** that establish the initial and final states of the animation's style, along with any potential intermediary points.



Style Declaration

- To define animation, the first part is style declaration which specifies the following properties:

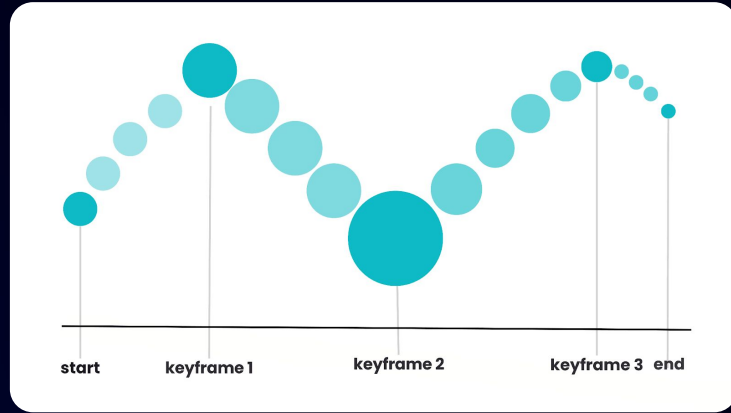


Animation Properties

Property	Description
<code>animation-name</code>	Specifies the name of the <code>@keyframes</code> animation to apply to the element.
<code>animation-duration</code>	Defines how long an animation should take to complete one cycle.
<code>animation-delay</code>	Sets a delay for the start of an animation.
<code>animation-iteration-count</code>	Specifies the number of times an animation cycle should be played.
<code>animation-direction</code>	Defines whether an animation should play in reverse on alternate cycles.
<code>animation-fill-mode</code>	Specifies what styles are applied to the element before and after the animation.
<code>animation-play-state</code>	Indicates whether the animation is running or paused.

Keyframes

- Specifies the animation code
- Once you have set up the basic animation properties of your animation,
- the next step is to specify its visual appearance.
- To achieve this, you must create one or multiple keyframes using the `@keyframes` rule.
- Each keyframe outlines the desired rendering of the animated element at a particular moment within the sequence of the animation.



Pop Quiz



Q. What is a keyframe animation in CSS?

A

A way to style text

B

A way to animate elements using
defined steps

Pop Quiz



Q. What is a keyframe animation in CSS?

A

A way to style text

B

A way to animate elements using
defined steps

Class Project

Build Responsive Mini Layout (Guided)

Summary

In this lecture, we have covered:

- Learned how to make web pages responsive across different devices
- Covered the basics of responsive design
- Understood media queries as a core skill
- Explored breakpoints and the mobile-first approach
- Got an introduction to CSS Grid for layout design
- Learned basic CSS transitions
- Understood the introduction to keyframe animations
- Built a responsive mini layout through a guided project

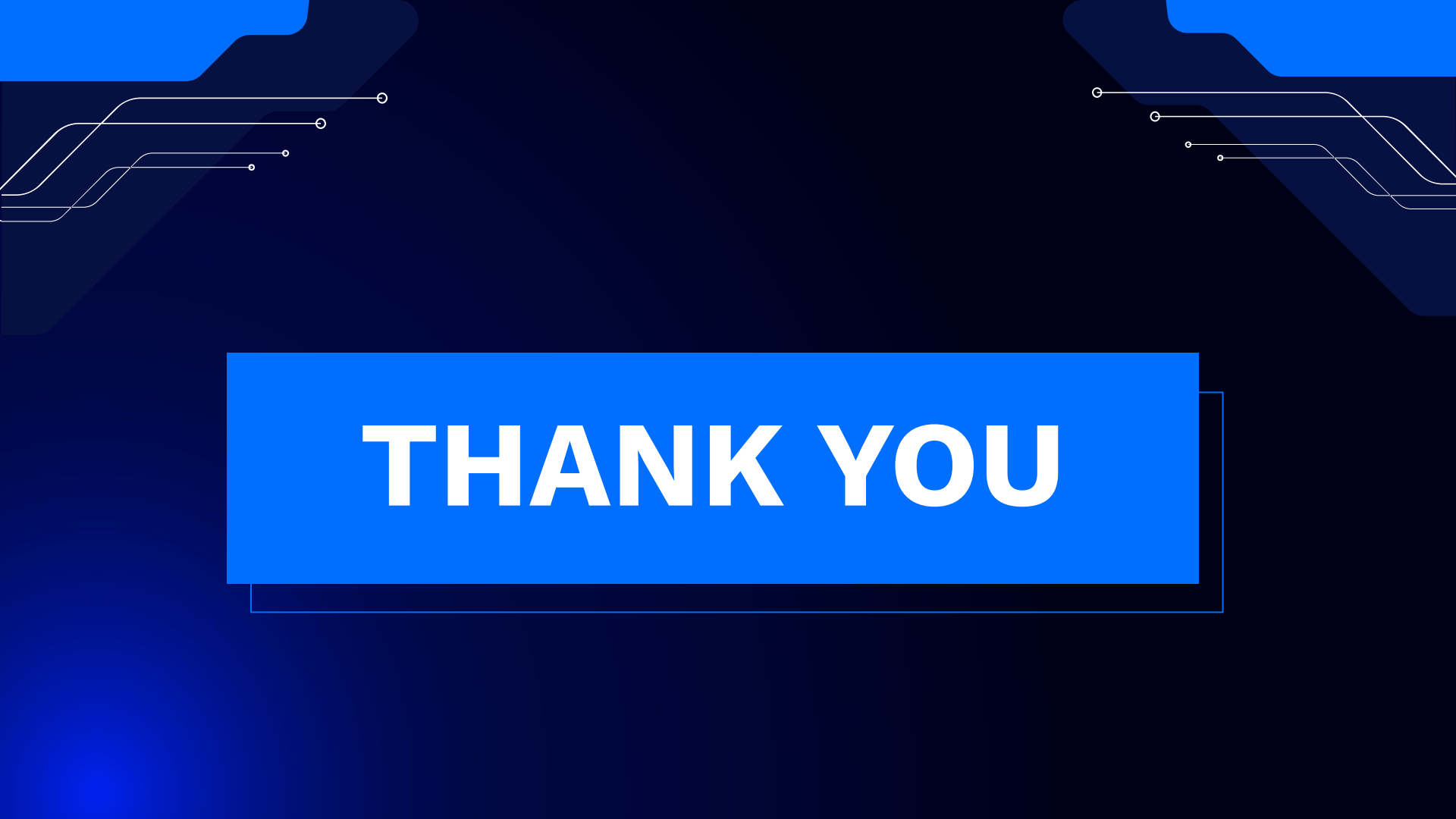




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU